🎟️ Event Ticket Booking Platform

🚀 Hackathon Project Documentation

📌 1. Project Overview

The **Event Ticket Booking Platform** is a full-stack application designed to manage the complete lifecycle of event ticketing, including:

- 👤 User Authentication & Role-Based Access
- 🎉 Event & Venue Management
- 💺 Seat Selection & Booking
- 🎟️ Ticket Generation
- 💰 Refund Handling
- 🆘 Support Case Management
- 🚪 Entry Validation System

The system ensures **strict lifecycle transitions**, preventing:

- Double booking
- Ticket reuse
- Unauthorized access
- Invalid refund processing

🏗️ 2. Tech Stack

🔷 Backend

- **FastAPI** (Async Python Framework)
- **MongoDB Atlas** (Cloud NoSQL Database)
- **Motor** (Async MongoDB Driver)
- **JWT Authentication**
- **Passlib (bcrypt)** for password hashing

🔷 Frontend

- Streamlit (Lightweight UI)

🔷 Dev Tools

- Git & GitHub
- Swagger UI for API Testing

🧠 3. System Architecture

```
Plain text
Frontend (Streamlit)
        ↓
FastAPI Routes
        ↓
Service Layer (Business Logic)
        ↓
MongoDB Atlas
```

Layer Responsibilities

Layer	Responsibility
Routes	API endpoints
Schemas	Request/Response validation
Services	Business logic
Models	MongoDB interaction
DB Layer	Connection & indexing

🔐 4. Authentication & Authorization

🔷 Features Implemented

- Secure Customer Signup
- Login with JWT token generation
- Password hashing (bcrypt)
- Role-based route protection
- OAuth2 Bearer token integration

🔷 Roles Supported

- Admin
- Organizer
- Customer
- Entry Manager
- Support Executive

🗄️ 5. Database Design (MongoDB Collections)

👤 Users

- name
- email
- hashed_password
- role
- created_at

🏢 Venues

- name
- location
- capacity

🎉 Events

- venue_id
- date
- category
- ticket_price
- status

💺 Seats

- event_id

- seat_number
- status (available / booked)

🛒 Bookings

- user_id
- event_id
- seat_ids
- status (pending / confirmed / cancelled)

🎟️ Tickets

- booking_id
- seat_id
- ticket_code (unique)
- status (active / used / cancelled)

💰 Refunds

- booking_id
- reason
- status (pending / approved / rejected)

🆘 Support Cases

- user_id
- description
- status

🚪 Entry Logs

- ticket_id
- validated_by
- validation_status
- entry_time

🔄 6. Booking Lifecycle

Step 1 — Customer selects seats

System verifies seat availability.

Step 2 — Booking created

Seats are temporarily locked.

Step 3 — Booking confirmed

- Seats → booked
- Tickets → generated
- Order → confirmed

Step 4 — Entry validation

- Ticket → used
- Entry log created

Step 5 — Refund (if approved)

- Order → refunded
- Seats → available
- Ticket → cancelled

🚫 7. Concurrency Protection

To prevent double booking:

```python
update_one(
  {"_id": seat_id, "status": "available"},
  {"$set": {"status": "booked"}}
)
```

If seat already booked → request fails.

⚡ 8. Key Features

✅ Async MongoDB operations
✅ Unique index on ticket_code
✅ Compound index on event_id + seat_number
✅ JWT secured endpoints
✅ Role validation
✅ Proper service-layer separation
✅ Production-style architecture

🧪 9. Testing Strategy

- Swagger UI for API validation
- MongoDB Atlas dashboard verification
- Role-based route testing
- Double booking simulation
- Ticket reuse prevention testing

📈 10. Indexing Strategy

To ensure performance and data integrity:

- `ticket_code` → Unique index
- `(event_id, seat_number)` → Compound unique index
- `user_id` → Indexed in support
- `ticket_id` → Indexed in entry logs

🚀 11. How To Run

Backend

```bash
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Access:

```
http://127.0.0.1:8000/docs
```

Conclusion

The system demonstrates:

- Clean backend architecture
- Secure authentication
- Strict state transitions
- Scalable MongoDB design
- Real-world event management flow

🎟️ **Event Ticket Booking Platform🚀 Hackathon Project Documentation📌 1. Project Overview**

The **Event Ticket Booking Platform** constitutes a full-stack application engineered to manage the comprehensive lifecycle of event ticketing, encompassing:

- User Authentication and Role-Based Access Control
- Event and Venue Management
- Advanced Seat Selection and Booking Mechanisms
- Automated Ticket Generation

- Efficient Refund Processing and Handling
- Structured Support Case Management
- Integrated Entry Validation System

The system strictly enforces **lifecycle transitions**, thereby mitigating the potential for:

- Double booking occurrences
- Unauthorized ticket reuse
- Unpermitted system access
- Invalid or improper refund processing

🏗️ 2. Technical Architecture🔷 Backend Components

- **FastAPI** (Asynchronous Python Web Framework)
- **MongoDB Atlas** (Cloud-hosted NoSQL Database Solution)
- **Motor** (Asynchronous MongoDB Driver)
- **JWT Authentication** Implementation
- **Passlib (bcrypt)** for Cryptographic Password Hashing

🔷 Frontend Interface

- Streamlit (Lightweight User Interface Framework)

🔷 Development Tools

- Git and GitHub for Version Control
- Swagger UI for API Endpoint Documentation and Testing

🧠 3. System ArchitectureLayer Responsibilities

Layer	Primary Responsibility
Routes	Definition of API endpoints
Schemas	Validation of Request and Response payloads
Services	Implementation of core business logic
Models	Abstraction for MongoDB data interaction

DB Layer	Management of database connection and indexing

🔐 4. Authentication and Authorization Module🔷 Implemented Features

- Secure Customer Registration Protocol
- Login functionality with JWT token generation
- Robust Password Hashing utilizing bcrypt
- Role-based access protection for routes
- Integration of OAuth2 Bearer token scheme

🔷 Supported Roles

- Administrator
- Organizer
- Customer
- Entry Manager
- Support Executive

🗄️ 5. Database Schema (MongoDB Collections)👤 Users Collection

- `name`
- `email`
- `hashed_password`
- `role`
- `created_at`

🏛️ Venues Collection

- `name`
- `location`
- `capacity`

🎉 Events Collection

- `venue_id` (Reference)
- `date`
- `category`
- `ticket_price`
- `status`

💺 Seats Collection

- `event_id` (Reference)
- `seat_number`
- `status` (available / booked)

🛒 **Bookings Collection**

- `user_id` (Reference)
- `event_id` (Reference)
- `seat_ids` (List of References)
- `status` (pending / confirmed / cancelled)

🎟️ **Tickets Collection**

- `booking_id` (Reference)
- `seat_id` (Reference)
- `ticket_code` (Unique Identifier)
- `status` (active / used / cancelled)

💰 **Refunds Collection**

- `booking_id` (Reference)
- `reason`
- `status` (pending / approved / rejected)

🆘 **Support Cases Collection**

- `user_id` (Reference)
- `description`
- `status`

🚪 **Entry Logs Collection**

- `ticket_id` (Reference)
- `validated_by` (Entry Manager ID)
- `validation_status`
- `entry_time`

🔄 **6. Booking Lifecycle ManagementStep 1 — Seat Selection**

The system verifies the current availability of selected seats.**Step 2 — Booking Initiation**

A booking record is created, and the selected seats are temporarily locked.**Step 3 — Booking Confirmation**

- Seat status is transitioned to 'booked'.
- Corresponding Tickets are generated.
- The Order status is set to 'confirmed'.

Step 4 — Entry Validation

- Ticket status is transitioned to 'used'.
- An Entry log record is created.

Step 5 — Refund Processing (upon approval)

- The Order status is updated to 'refunded'.
- Seat status is reverted to 'available'.
- Ticket status is set to 'cancelled'.

🚫 7. Concurrency Protection Mechanism

To effectively prevent double booking scenarios:

- *(Content Missing in Original - Placeholder)*

⚡ 8. Key System Capabilities

- Asynchronous MongoDB database operations
- Enforcement of a unique index on the `ticket_code` field
- Implementation of a compound unique index on `event_id` and `seat_number`
- Security for API endpoints via JWT
- Robust Role validation for access control
- Adherence to proper service-layer separation principles
- Utilization of a Production-style architecture

🧪 9. Testing and Quality Assurance Strategy

- Utilization of Swagger UI for API functional validation
- Verification via the MongoDB Atlas dashboard
- Systematic testing of role-based route protection
- Controlled simulation of double booking attempts
- Testing protocol for ticket reuse prevention

📈 10. Database Indexing Strategy

To optimize performance and maintain data integrity, the following indices are implemented:

- `ticket_code`: Unique Index
- (`event_id`, `seat_number`): Compound Unique Index
- `user_id`: Indexed within the Support collection
- `ticket_id`: Indexed within the Entry Logs collection

🚀 11. Deployment and Execution InstructionsBackend

- *(Content Missing in Original - Placeholder)*

Conclusion

The implemented system successfully demonstrates the following attributes:

- A clean and modular backend architecture
- Secure and reliable authentication implementation
- Strict adherence to defined state transition rules
- A scalable MongoDB database design
- Accurate representation of a real-world event management workflow